

Parking Rental System — Agent Development Context

This document is the complete specification for building the Parking Rental System. It is written to be given to a coding agent (or a new developer) as full context — read it end to end before writing any code.

1. Project Objective

Build a full-stack parking rental platform where:

- **Owners** list parking spaces (location, price, availability) and manage bookings.
- **Renters** search available spaces by location/date, book a slot, and pay online.
- **Admins** oversee users, listings, bookings, and transactions.

Architecture: Java Spring Boot microservices behind an API Gateway, service discovery via Eureka, JWT-based auth (RS256, access + refresh tokens), MySQL (database-per-service), and a React frontend. Notification service is **out of scope** for this build.

2. Services in scope

Service	Port	Owns
eureka-server	8761	Service registry
common-security (library, not run) Shared JWT filter/validator		
api-gateway	8080	Routing + JWT pre-filter, CORS
auth-service	8081	Users, JWT issuing/refresh/revoke
user-service	8082	Profiles, Vehicles
parking-service	8083	Listings, availability
booking-service	8084	Bookings, status history
payment-service	8085	Transactions, refunds, webhook
frontend	5173 (dev)	React SPA

All client traffic goes through the gateway at http://localhost:8080.

3. Tech stack

Backend: Java 17, Spring Boot 3.3.x, Spring Cloud 2023.0.3 (Gateway, Eureka, OpenFeign), Spring Security, Spring Data JPA, MySQL 8, JWT 0.12.6 (RS256), Flyway, Lombok, Resilience4j.

Frontend: React (Vite), React Router, Axios, Redux Toolkit or Context API, React Hook Form + Yup, Stripe.js (or Razorpay SDK), Tailwind or MUI.

Infra: Docker Compose, Nginx (serving the React build).

4. Root project structure

parking-rental-system/

├── pom.xml # optional Maven reactor parent (see note below)

├── docker-compose.yml

├── common-security/

├── eureka-server/

├── api-gateway/

├── auth-service/

├── user-service/

├── parking-service/

├── booking-service/

├── payment-service/

└── frontend/

Note on root pom.xml: a multi-module Maven reactor is optional. Each service can instead keep its own standalone pom.xml extending spring-boot-starter-parent directly and be built independently (cd <service> && mvn clean install). If skipped, common-security must still be built and mvn installed locally before any dependent service builds, since they reference it via <dependency>.

Each service follows this internal shape:

<service>/

└─ pom.xml

└─ src/main/

└─ java/com/parking/<service_name>/

| └─ <Service>Application.java

| └─ config/ (SecurityConfig, etc.)

| └─ controller/

| └─ service/ + serviceImpl/ (or combined)

| └─ repository/

| └─ model/

| └─ dto/

| └─ client/ (Feign clients, if any)

| └─ exception/

└─ resources/

└─ application.yml

common-security (package com.parking.common.security) contains only: JwtProperties, JwtValidator, JwtAuthenticationFilter, SecurityConstants. It has

<packaging>jar</packaging>, no main() class, and its spring-boot-maven-plugin must have <skip>true</skip> configured (it is a library, not a runnable app — repackaging will fail otherwise since there's no main class).

5. Database — one schema per service (MySQL)

- auth_db: users(id, email, password_hash, role, enabled, created_at), refresh_tokens(id, user_id, token_hash, expires_at, revoked), revoked_tokens(jti, expires_at)
- user_db: user_profiles(id, user_id, name, phone, address), vehicles(id, plate_no, type, make, model, color, user_profile_id)

- parking_db: parking_spaces(id, owner_id, address, lat, lng, price_per_hour, description), availability_slots(id, space_id, start_time, end_time, is_booked)
- booking_db: bookings(id, renter_id, space_id, slot_id, vehicle_id, status, total_price, created_at), booking_status_history(id, booking_id, status, changed_at)
- payment_db: transactions(id, booking_id, amount, status, gateway_ref, created_at), refunds(id, transaction_id, amount, status)

Use spring.jpa.hibernate.ddl-auto: update for development; use Flyway migrations for anything closer to production.

6. JWT security design (implemented once in common-security, reused everywhere)

- **Auth Service** signs tokens with an **RS256 private key**; every other service verifies with the paired **public key** (set as jwt.public-key in each service's application.yml, same value everywhere).
 - **Access token**: 15 min TTL, claims = sub (userId), role, jti (unique id).
 - **Refresh token**: 7 day TTL, stored hashed in refresh_tokens, delivered as an HttpOnly Secure cookie (not accessible to JS).
 - **Logout**: refresh token marked revoked; access token's jti added to revoked_tokens blacklist.
 - **Gateway** validates the token before routing (defense in depth); **every downstream service also validates independently** (zero-trust) via its own copy of the JwtAuthenticationFilter imported from common-security.
 - Role checks in controllers via @PreAuthorize("hasRole('OWNER')") etc., enabled with @EnableMethodSecurity in each service's SecurityConfig.
 - Passwords hashed with BCrypt.
-

7. API Contract

Base URL (via gateway): http://localhost:8080. All authenticated calls send Authorization: Bearer <accessToken>.

Standard response envelope:

```
{ "success": true, "data": { }, "error": null }
```

Error shape:

```
{ "success": false, "data": null, "error": { "code": "VALIDATION_ERROR", "message": "..." } }
```

Auth Service — /api/auth

Method	Path	Request body	Response data
POST	/register	{ email, password, role }	{ userId, email, role }
POST	/login	{ email, password }	{ accessToken, expiresIn, userId, role } (+ refresh cookie)
POST	/refresh	— (cookie)	{ accessToken, expiresIn }
POST	/logout	—	null
GET	/me	—	{ userId, email, role }

User Service — /api/users

Method	Path	Request body	Response data
GET	/userId	—	{ id, userId, name, phone, address }
PUT	/userId	{ name, phone, address }	same shape
POST	/userId/vehicles	{ plateNo, type, make, model, color }	{ id, plateNo, type }
GET	/userId/vehicles	—	[{ id, plateNo, type }]
DELETE	/userId/vehicles/{vehicleId}	—	null

Parking Service — /api/parking

Method	Path	Request body	Response data
POST	/ (OWNER)	{ address, lat, lng, pricePerHour }	listing object

Method	Path	Request body	Response data
		{ id, description }	
GET	/ {id}	—	listing object
PUT	/ {id} (OWNER)	same as POST	listing object
DELETE	/ {id} (OWNER)	—	null
GET	/ ?lat&lng&radiusKm&date&startTime&endTime&page&size	—	paginated listing search results
GET	/owner/{ownerId}	—	[listing]
POST	/ {id}/availability	{ startTime, endTime }	slot object
GET	/ {id}/availability?from&to	—	[slot]
PUT	/slots/{slotId}/mark-booked	— (internal, called by booking-service)	null
PUT	/slots/{slotId}/mark-unbooked	— (internal)	null

Booking Service — /api/bookings

Method	Path	Request body	Response data
POST	/ ?renterId=	{ spaceId, slotId, vehicleId, totalPrice }	{ id, status, totalPrice }
GET	/ {id}	—	booking object

Method	Path	Request body	Response data
GET	/renter/{renterId}	—	[booking]
PATCH	/id/cancel	—	booking object
PATCH	/id/status?status=	— (internal, called by payment-service)	null

Payment Service — /api/payments

Method	Path	Request body	Response data
POST	/initiate	{ bookingId, amount, currency }	{ transactionId, status, gatewayRef }
POST	/webhook	raw gateway payload, signature-verified, unauthenticated	{ received: true }
GET	/id	—	transaction object
GET	/booking/{bookingId}	—	[transaction]
POST	/id/refund	{ amount, reason }	{ refundId, status }

8. Backend build order (do in this sequence)

1. **common-security** — JWT validator + filter. Build and mvn install first.
2. **eureka-server** — bare registry, confirm dashboard loads.
3. **auth-service** — full token lifecycle (issue, refresh, revoke), test in Postman standalone.
4. **api-gateway** — routes to all services by path, JWT pre-filter, CORS for the React origin.
5. **user-service** — profile + vehicle CRUD (Vehicle is a separate @Entity linked @ManyToOne to UserProfile, not embedded).
6. **parking-service** — listing CRUD, availability slots, search.
7. **booking-service** — booking creation with slot-conflict check via Feign call to parking-service; status lifecycle.

8. **payment-service** — payment initiation, webhook handling, refunds; Feign call to booking-service to confirm/fail bookings after payment.
9. Wire booking-service ↔ parking-service and payment-service ↔ booking-service via **OpenFeign** clients (@FeignClient(name = "<service-name>"), resolved through Eureka).
10. Add GlobalExceptionHandler (@RestControllerAdvice) per service returning the standard error envelope.
11. Bring everything up together with docker-compose up and walk through: register → login → create listing → search → book → pay (test mode) → confirm booking status updates.

Each entity gets: model (JPA @Entity) → repository (JpaRepository) → service interface + impl → controller → security config (import JwtAuthenticationFilter/JwtValidator from common-security, wire into a SecurityFilterChain bean with @EnableMethodSecurity).

9. Frontend build order

Build in this sequence so each layer has something to test against:

1. Scaffold Vite + React project; install axios, react-router-dom, a state solution, form library, Stripe SDK.
2. **api/axiosInstance.js** — base client with an interceptor that attaches the access token and silently calls /api/auth/refresh on a 401, then retries the original request.
3. **Auth context/store** — holds { user, accessToken }, exposes login(), logout(), refresh().
4. **Login / Register pages** — wire to Auth Service endpoints.
5. **ProtectedRoute** + role-aware **AppRoutes** — gate pages by auth state and role.
6. **NavBar** — links change based on role (renter/owner/admin).
7. **Search/listing pages** — SearchListingsPage (filters: location, date range, price) + ParkingCard; ListingDetailPage with an availability calendar.
8. **Booking + payment flow** — BookingCheckoutPage: create booking → initiate payment → Stripe Elements confirm → poll or redirect to a confirmation page.
9. **MyBookingsPage** — renter's booking history with cancel action.

10. **Owner pages** — OwnerDashboardPage, MyListingsPage, CreateEditListingPage (with availability slot editor), IncomingBookingsPage.

11. **Admin pages** (if in scope) — AdminUsersPage, AdminBookingsPage, AdminTransactionsPage.

12. Polish: loading states, toast notifications, form validation everywhere, empty states.

Match every API call's request/response shape exactly to Section 7 — this is the contract both frontend and backend teams build against independently.

10. Cross-cutting rules for whoever/whatever writes the code

- Never put a plaintext password or private key in a pom.xml, application.yml committed to git, or frontend code — use environment variables (`${DB_PASSWORD}`, `${JWT_PUBLIC_KEY}`, `${STRIPE_SECRET_KEY}`).
- Every protected controller method must check role via `@PreAuthorize`, not just rely on the filter authenticating a user.
- payment-service's webhook endpoint must be excluded from the JWT filter chain (`permitAll()` on that specific path) and instead validated via gateway signature verification — it's called by Stripe/Razorpay, not by a logged-in user.
- Services never call each other's databases directly — only via REST (OpenFeign) through Eureka-resolved service names.
- Keep DTOs (request/response records) separate from JPA entities — never return an `@Entity` directly from a controller.
- All list endpoints that could grow large (search, booking history) should be paginated.
- Match field names exactly between backend DTOs and frontend Axios calls — this is the #1 source of integration bugs across a multi-person team.